MANCHESTER 1824
The University of Manchester

Energy-Efficient UAV Path Planning in IOT Networks: A Deep Reinforcement Learning Aided Approach

A dissertation submitted to The University of Manchester for the degree of

Bachelors of Engineering in Electrical and Electronics Engineering

in the Faculty of Science and Engineering

Year of submission

2026

Student ID

11085980

School of Engineering

Terms and abbreviations

Acronym	Definition
UAV	Unmanned Aerial Vehicle
IoT	Internet of Things
RL	Reinforcement Learning
DRL	Deep Reinforcement Learning
DQN	Deep Q-Network
PPO	Proximal Policy Optimization
MDP	Markov Decision Process
LoRa	Long Range
LoRaWAN	Long Range Wide Area Network
SF	Spreading Factor
ADR	Adaptive Data Rate
RSSI	Received Signal Strength Indicator
SNR	Signal-to-Noise Ratio
PHY	Physical Layer
MAC	Medium Access Control
AoI	Age of Information
DR	Domain Randomization
GPS	Global Positioning System
AoI	Age of Information
ToA	Time on Air
GtA	Ground-to-Air
LoS	Line-of-Sight
NLOS	Non-Line-of-Sight
2D	Two-Dimensional
3D	Three-Dimensional

Abstract

This dissertation addresses energy-efficient, dynamic path planning for Unmanned Aerial Vehicles (UAVs) tasked with data collection in unsynchronized LoRaWAN Internet of Things (IoT) networks. We propose a protocol-aware Deep Reinforcement Learning (DRL) framework, utilizing a Deep Q-Network (DQN) agent trained within a custom, high-fidelity Gymnasium environment. The environment models the UAV's three-dimensional position as a direct control input, incorporating realistic LoRa (Long Range) physics, including the Two-Ray Ground Reflection path loss, log-normal shadowing, asynchronous duty cycles, and Adaptive Data Rate (ADR) latency. The DQN policy optimizes a multi-objective reward function to balance energy consumption, data throughput, and Age of Information (AoI). Evaluation across dynamic, multi-sensor scenarios demonstrates strong policy generation that prevents the "Spreading Factor Monoculture" congestion. Comparative analysis shows that the DRL agent delivers superior network throughput and true energy efficiency compared to baseline greedy heuristics

Declaration of originality

I hereby confirm that no portion of the work referred to in the thesis has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright statement

i The author of this thesis (including any appendices and/or schedules to this thesis) owns certain copyright or related rights in it (the “Copyright”) and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.

ii Copies of this thesis, either in full or in extracts and whether in hard or electronic copy, may be made *only* in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.

iii The ownership of certain Copyright, patents, designs, trademarks and other intellectual property (the “Intellectual Property”) and any reproductions of copyright works in the thesis, for example graphs and tables (“Reproductions”), which may be described in this thesis, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.

iv Further information on the conditions under which disclosure, publication and commercialisation of this thesis, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see `http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420`), in any relevant Thesis restriction declarations deposited in the University Library, The University Library’s regulations (see `http://www.library.manchester.ac.uk/about/regulations/`) and in The University’s policy on Presentation of Theses.

Acknowledgments

I am deeply grateful to my supervisor Dr. Zahra Mobini, for her emaculate guidance and support throughout the project.

I would like to thank my family - without their support none of this would have been possible.

1 Introduction

1.1 Background and motivation

The deployment of IoT sensor networks has garnered significant research attention due to its wide-ranging applications in smart agriculture, environmental monitoring, and disaster response [1]. In many of these scenarios, fixed infrastructure is either unavailable or prohibitively expensive to deploy. Consequently, UAVs have emerged as a critical enabler for data collection, offering the flexibility to serve as mobile gateways that can be deployed on-demand to remote or hazardous locations [1]. By combining the long-range capabilities of the LoRaWAN protocol with the mobility of UAVs, it is theoretically possible to collect data from vast sensor fields with minimal infrastructure [2].

However, integrating mobile UAVs into dense LoRaWAN networks presents fundamental challenges that standard protocols were not designed to handle. The LoRaWAN architecture, specifically its ADR mechanism, is engineered for static, ground-based networks. As identified by Zorbas et al. (2025), in dense deployments where link qualities change rapidly—such as with a moving UAV—the standard network management fails. Devices often converge to the same transmission settings, leading to a high rate of intra-SF collisions" and a degradation of network performance [3]. This phenomenon, often referred to as the SF12 Well" or "SF Monoculture," causes a congestion collapse where the network's concurrent channel capacity is wasted, and packet delivery rates plummet.

Existing solutions to this problem remain siloed. Standard path planning heuristics (such as lawnmower" patterns) optimize solely for flight distance, ignoring the underlying telecommunications physics [4]. These protocol-blind" trajectories inadvertently force sensors into the SF Monoculture, causing the UAV to hover inefficiently while waiting for data that cannot be decoded due to collisions. Conversely, mathematical optimization approaches (like Mixed Integer Linear Programming) can solve the resource allocation problem for static networks but lack the real-time adaptability required for dynamic, mobile environments [3].

To address these limitations, a protocol-aware control framework is required—one that integrates the UAV's physical mobility with the logical constraints of the LoRaWAN protocol. Recent literature has explicitly suggested that reinforcement learning models could be employed to adapt to these evolving network conditions [3], [5]. By treating the UAV's 3D posi-

tion as a direct control input for the radio protocol, an intelligent agent can proactively manage the RF environment, breaking the SF monoculture and ensuring true energy efficiency through optimized network throughput.

1.2 Aims and objectives

The primary aim of this project is to develop a protocol-aware Deep Reinforcement Learning (DRL) framework capable of optimizing UAV flight trajectories for energy-efficient data collection in dense, unsynchronized LoRaWAN networks. The goal is to demonstrate that an intelligent agent can resolve the “Spreading Factor (SF) Monoculture” bottleneck by using its physical position as a direct control input for the radio protocol.

Specifically, the project will train a Deep Q-Network (DQN) agent within a high-fidelity simulation to learn a non-obvious, network-aware policy that maximizes data throughput while minimizing energy consumption.

The main objectives of the project are summarized below:

- To develop a high-fidelity simulation environment compatible with the Gymnasium interface, incorporating rigorous telecommunications physics including the Two-Ray Ground Reflection path loss model, log-normal shadowing, and asynchronous duty cycles.
- To model the specific constraints of the LoRaWAN protocol, particularly the Adaptive Data Rate (ADR) latency, to replicate the causal link between UAV positioning, RSSI, and Spreading Factor allocation.
- To formulate the UAV data collection problem as a Markov Decision Process (MDP), designing a multi-objective reward function that balances the competing goals of energy conservation, data freshness (AoI), and collection volume.
- To train a Deep Q-Network (DQN) agent to learn an emergent flight policy that autonomously navigates the “Starvation vs. Throughput” trade-off without human intervention.
- To evaluate the performance of the protocol-aware DRL agent by comparing it against standard greedy path-planning heuristics (e.g., “lawnmower” patterns) in terms of energy efficiency, network capacity, and successful packet delivery.

- To assess the trade-offs between computational complexity, policy robustness, and real-world feasibility, identifying the “Sim-to-Real” gap as a key area for future research.

In this project, the collective goal is to demonstrate that a protocol-aware agent is capable of generating optimal, energy-efficient trajectories that satisfy network fairness and throughput requirements.

1.3 Report structure

This report is organised as follows. Section 2 reviews existing literature on UAV-assisted data collection, LoRaWAN scalability challenges, and the application of Deep Reinforcement Learning (DRL) in wireless networks. Section 3 discusses preliminary concepts fundamental to this work, including the LoRaWAN physical layer, the Two-Ray Ground Reflection model, and the fundamentals of Q-Learning.

The project’s methodology is detailed in Section 4, which presents the system model, the custom Gymnasium simulation environment, and the formulation of the DRL agent’s state space and reward function. The simulation results and a critical discussion of the agent’s performance against greedy baselines are presented in Section 5. Finally, the conclusion and a consideration of future research avenues—specifically regarding generalization and the “Sim-to-Real” gap—can be found in Section 6.

2 Literature review

The integration of Unmanned Aerial Vehicles (UAVs) into Internet of Things (IoT) ecosystems has revolutionized data collection in remote and hazardous environments. As detailed in the comprehensive survey by Wei et al. [1], UAV-assisted data collection offers distinct advantages over static infrastructure, including flexible deployment, line-of-sight communication links, and the ability to service sparse sensor networks on demand. However, the transition from static to mobile gateways introduces complex challenges in network management and protocol stability.

2.1 LoRaWAN Scalability and the “SF Monoculture”

While LoRaWAN is the de facto standard for long-range, low-power IoT, its performance degrades significantly in dense or mobile scenarios. Nakamura et al. [2] highlight the necessity of robust protocols for connecting edge computing nodes, emphasizing that reliable transfer of “small data” requires careful management of connection setups and message lengths. A critical failure mode in these networks is the congestion collapse known as the “Spreading Factor (SF) Monoculture” or “SF12 Well.” Zorbas et al. [3] provide a rigorous analysis of this phenomenon, demonstrating that standard Adaptive Data Rate (ADR) mechanisms often fail in dense deployments, causing devices to converge on the slowest transmission settings (SF12). This results in a high rate of intra-SF collisions and wasted channel capacity, a bottleneck that standard path planning heuristics fail to address.

2.2 Existing Optimization Approaches

Current approaches to optimizing UAV-LoRaWAN networks generally fall into two categories: static mathematical optimization and energy-focused path planning.

In the domain of mathematical optimization, Zorbas et al. [3] propose a Mixed Integer Linear Programming (MILP) framework to determine the optimal SF distribution that maximizes packet reception probability. While this method provides a theoretically optimal allocation for static networks, it lacks the real-time adaptability required for dynamic UAV missions where the channel conditions change continuously.

In the domain of path planning, Xiong et al. [4] introduce “FlyingLoRa,” a scheme designed to minimize energy consumption by jointly optimizing the UAV’s 3D trajectory and transmission scheduling. This approach represents the state-of-the-art in energy-efficient collection, utilizing iterative algorithms to find sub-optimal solutions for flight paths. However, such methods often treat the communication protocol as a static constraint rather than a dynamic variable that the UAV can actively manipulate.

2.3 The Shift to AI-Native Networking

To overcome the limitations of static and heuristic methods, the field is moving toward intelligent, autonomous network management. Nguyen et al. [5] outline the vision for 6G IoT, predicting a shift toward “AI-native” networks where Deep Reinforcement Learning (DRL) and edge intelligence play a central role in managing massive connectivity and ultra-reliable low-latency communications. This paradigm suggests that the next generation of network optimizers will not be static algorithms, but autonomous agents capable of learning complex policies from high-dimensional state spaces.

2.4 Summary and Project Positioning

Despite significant progress in UAV path planning, a gap remains in effectively coupling physical mobility with protocol-level optimization. Existing works either optimize the path while ignoring the protocol’s failure modes (creating the SF Monoculture) or optimize the protocol using static math that cannot adapt to mobility. Explicitly acknowledging this gap, Zorbas et al. [3] conclude that “reinforcement learning models could be employed to adapt over time to evolving network conditions.”

In this report, the focus is placed on this specific intersection: using a DRL agent to solve the SF Monoculture. Unlike the static optimization in [3] or the energy-centric heuristics in [4], the proposed framework treats the UAV’s physical position as a direct control input for the LoRaWAN protocol, allowing the agent to learn an emergent policy that balances energy efficiency with rigorous protocol health.

3 Preliminaries

This section reviews the fundamental concepts and mathematical models that underpin the proposed framework. Specifically, we detail the physical layer characteristics of the LoRaWAN protocol, the radio propagation models used to simulate the UAV-to-ground channel, and the theoretical foundations of Deep Reinforcement Learning (DRL). The proposed architecture employs a Deep Q-Network (DQN) agent acting within a high-fidelity environment; this necessitates a rigorous definition of both the telecommunications physics that

govern the environment's state transitions and the learning mechanisms that drive the agent's decision-making process.

3.1 LoRaWAN Physical Layer

LoRa (Long Range) is a proprietary spread-spectrum modulation technique derived from Chirp Spread Spectrum (CSS). It encodes information by varying the frequency of "chirps" over time, which provides exceptional resilience to interference and noise [1].

3.1.1 Spreading Factors and Orthogonality

The key parameter in LoRa modulation is the Spreading Factor (SF), which determines the duration of a chirp. LoRaWAN defines six orthogonal spreading factors, $SF \in \{7, 8, \ldots, 12\}$.

- **SF7:** Lowest range, highest data rate, shortest Time-on-Air.
- **SF12:** Longest range, lowest data rate, longest Time-on-Air.

A critical feature of LoRa is the **orthogonality** of these spreading factors. Signals modulated with different SFs can theoretically be received simultaneously by a gateway on the same frequency channel without interfering with each other [3]. This property creates the "concurrent channel capacity" that this project aims to exploit.

3.1.2 The Capture Effect

In dense networks, collisions occur when two devices transmit on the same SF and frequency simultaneously. However, LoRa receivers exhibit the **Capture Effect**: if one signal is sufficiently stronger than the interfering signal (typically by a margin of 6 dB), the receiver can successfully decode the stronger packet while suppressing the weaker one [3]. This physical phenomenon is modeled in the simulation to resolve co-SF collisions.

3.2 Channel Propagation Model

To accurately simulate the signal strength (RSSI) received by the UAV, a simple Free Space Path Loss (FSPL) model is insufficient. This project employs the **Two-Ray Ground Reflection Model**, which accounts for both the direct Line-of-Sight (LoS) path and the path reflected off the ground.

The received power P_r at distance d is approximated by:

$$P_r = P_t G_t G_r \frac{h_t^2 h_r^2}{d^4} \tag{1}$$

where P_t is the transmit power, G_t and G_r are antenna gains, and h_t and h_r are the heights of the transmitter (sensor) and receiver (UAV). Unlike the $1/d^2$ decay of free space, the Two-Ray model predicts a steeper $1/d^4$ decay at long distances, providing a more realistic constraint for the UAV's data collection radius.

3.3 Deep Reinforcement Learning

Reinforcement Learning (RL) formalizes the decision-making problem as a Markov Decision Process (MDP), defined by the tuple (S, A, P, R, γ), where S is the state space, A is the action space, P is the transition probability, R is the reward function, and γ is the discount factor.

3.3.1 Deep Q-Networks (DQN)

Standard Q-Learning stores state-action values in a table, which is computationally intractable for continuous or high-dimensional state spaces (such as a UAV's position). A **Deep Q-Network (DQN)** solves this by using a neural network to approximate the Q-value function $Q(s, a; \theta)$, where θ represents the network weights. The agent minimizes the temporal difference error defined by the Bellman equation:

$$L(\theta) = \mathbb{E}\left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\right)^2\right] \tag{2}$$

This project utilizes a DQN agent to learn the non-linear mapping between the complex network state (RSSI, AoI, battery) and the optimal flight vector.

4 Methodology

This chapter details the development of the protocol-aware Deep Reinforcement Learning (DRL) framework. We first present the system model, which mathematically defines the UAV's kinematics, energy consumption, and the stochastic LoRaWAN channel. Subsequently, we formulate the data collection task as a Markov Decision Process (MDP), defining the specific state space, action space, and multi-objective reward function used to train the agent. Finally, we describe the algorithmic implementation of the Deep Q-Network (DQN) within the custom Gymnasium simulation environment.

4.1 System Model

We consider a remote IoT monitoring environment comprising a set $\mathcal{N} = \{1, \ldots, N\}$ of static sensor devices uniformly distributed over a rectangular Region of Interest (RoI) $\mathcal{W} \subset \mathbb{R}^2$. These devices continuously generate monitoring data, which accumulates in finite transmission buffers. Unlike simple deadline-based models, we assume data carries value only if it is collected before the buffer capacity B_{max} is exceeded, after which packet loss occurs.

The system model is characterized by a rotary-wing UAV acting as a mobile LoRaWAN gateway, dispatched to harvest data from the ground sensors. The mission duration is fixed to a finite horizon T and divided into discrete time steps $t = 0, 1, \ldots, T$, each of duration Δt. We assume the UAV flies at a fixed altitude h to maintain a consistent ground-to-air channel geometry, updating its horizontal position $p_t = (x_t, y_t)$ at each step. Critically, the system employs the LoRaWAN protocol with orthogonal Spreading Factors (SF), allowing multiple sensors to upload data simultaneously provided they are assigned distinct SFs by the Adaptive Data Rate (ADR) mechanism. The UAV must therefore optimize its trajectory not merely to minimize distance, but to actively manage these spectral assignments to prevent network congestion.

4.1.1 UAV Kinematics and Energy Consumption

We consider a rotary-wing Unmanned Aerial Vehicle (UAV) operating within a discretized 3D grid space $\mathcal{W} \subset \mathbb{R}^3$. The mission is defined over a finite time horizon T, discretized into

time steps of duration Δt. At each time step k, the UAV's kinematic state is defined by its 3D position $p_k = [x_k, y_k, z_k]^T$.

The UAV is controlled by a discrete action space $\mathcal{A}$ consisting of five high-level motion primitives:

$$\mathcal{A} = \{\text{Move}_N, \text{Move}_S, \text{Move}_E, \text{Move}_W, \text{Hover}\} \tag{3}$$

Each movement action updates the UAV's position by a fixed step size δ_d in the corresponding cardinal direction. The `Hover` action maintains the current position $p_{k+1} = p_k$, which is critical for stabilizing the LoRaWAN link to facilitate data packet reception.

The energy consumption model is state-dependent. Let E_k denote the remaining battery energy at step k. The energy evolution is governed by:

$$E_{k+1} = E_k - P(a_k) \cdot \Delta t \tag{4}$$

where $P(a_k)$ is the instantaneous power consumption. We define distinct power profiles for flight modes:

$$P(a_k) = \begin{cases} P_{hover} & \text{if } a_k = \text{Hover} \\ P_{flight} & \text{if } a_k \in \{\text{Move}\} \end{cases} \tag{5}$$

Consistent with rotary-wing aerodynamics, we model the power consumption such that $P_{hover} > P_{flight}$ due to the lack of translational lift in static flight1. Accordingly, we assign a higher power cost to the Hover action (700 W) compared to translational Move actions (500 W). This power differential incentivizes the agent to optimize for efficient flight trajectories and penalizes excessive loitering, thereby aligning the reward signal with the physical reality of limited onboard energy storage.s

4.1.2 LoRaWAN Channel and Protocol Modeling

To accurately simulate the ground-to-air communication channel, a simple Free Space Path Loss (FSPL) model is insufficient as it neglects ground reflections, which are significant at low UAV altitudes. Instead, this work employs the **Two-Ray Ground Reflection Model**.

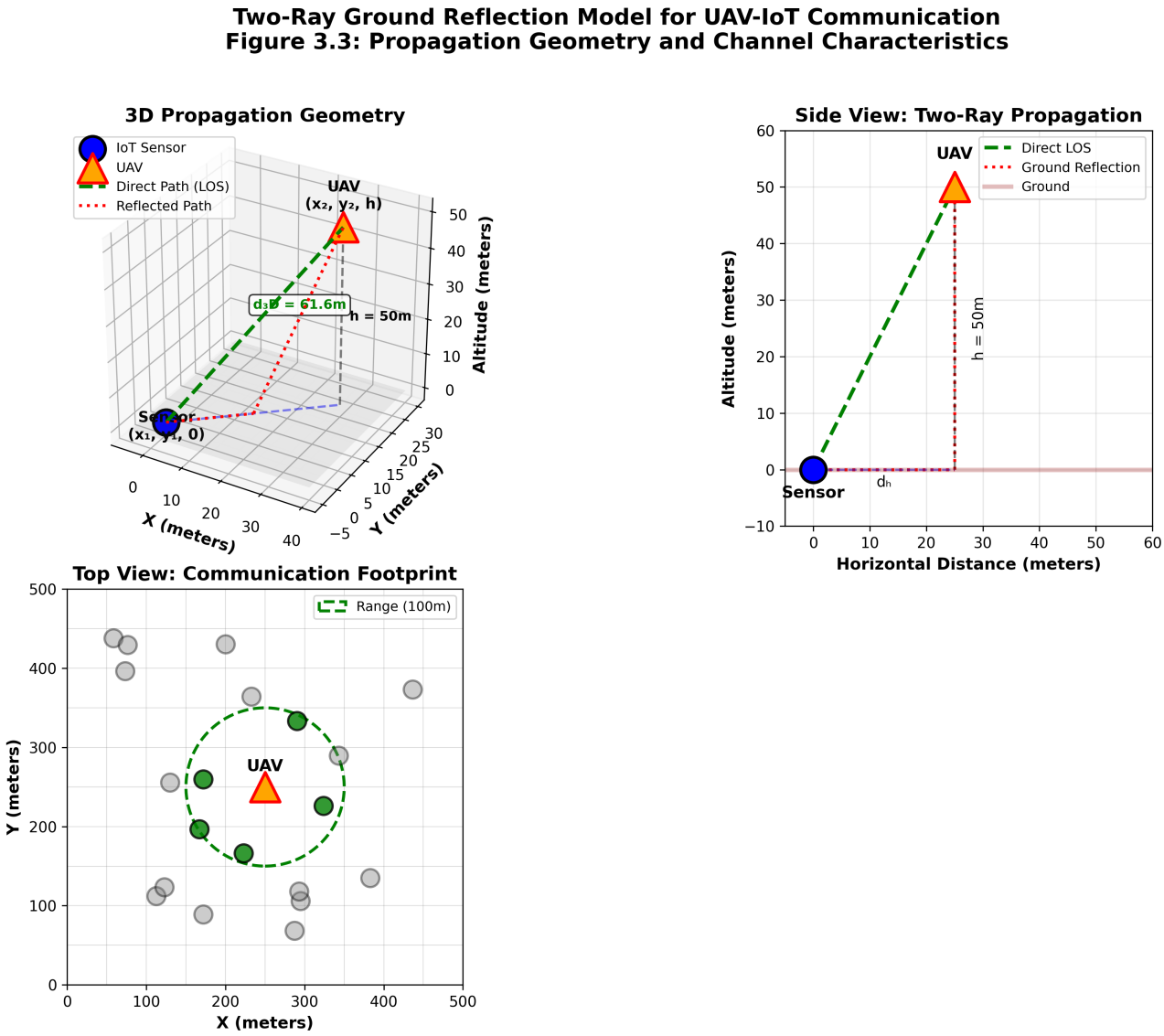

Fig. 1. Two-Ray Ground Reflection Model for UAV-IoT Communication. The diagram illustrates the 3D propagation geometry, distinguishing between the direct Line-of-Sight (LoS) path (green dashed line) and the ground-reflected path (red dotted line). This geometric modeling is critical for calculating accurate RSSI values and determining the appropriate Spreading Factor (SF) in the simulation.

As shown in Figure 1, the model calculates the path loss based on the interference between the direct and reflected rays. The received power P_r is given by:

$$P_r = P_t G_t G_r \frac{h_t^2 h_r^2}{d^4} \tag{6}$$

where h_t and h_r represent the heights of the transmitter (sensor) and receiver (UAV), respectively. This results in a steeper signal decay ($1/d^4$) compared to free space, enforcing realistic constraints on the UAV's data collection radius.

4.1.3 Distance and Propagation Geometry

The simulation operates in a 3D Cartesian coordinate system where the UAV's position at time t is denoted as $p_{u,t} = (x_u, y_u, h_u)$ and the i-th sensor's position is fixed at $p_{s,i} = (x_i, y_i, 0)$.

The Euclidean distance $d_{i,t}$ between the UAV and the i-th sensor is calculated as the 3D slant range:

$$d_{i,t} = \sqrt{(x_u - x_i)^2 + (y_u - y_i)^2 + h_u^2} \tag{7}$$

where h_u is the UAV's altitude. This 3D distance is critical for the **Two-Ray Ground Reflection Model**, as the path loss depends heavily on the height differential between the transmitter and receiver.

In the Two-Ray model, the phase difference between the direct Line-of-Sight (LoS) path and the ground-reflected path is a function of this distance and the reflection coefficient. For small grazing angles (typical for UAVs flying at altitude h_u collecting from ground sensors), the path loss L simplifies to the fourth-power law:

$$L \propto \frac{h_u^2 h_s^2}{d_{i,t}^4} \tag{8}$$

This quartic decay ($1/d^4$) means that small changes in the UAV's position or altitude have a magnified impact on the Received Signal Strength Indicator (RSSI), creating a highly sensitive environment that the DRL agent must learn to navigate precisely.

4.1.4 LoRaWAN Device Class and Traffic Model

We model the IoT sensor nodes as *LoRaWAN Class A devices*, operating in an asynchronous, sensor-initiated transmission mode. Class A operation imposes a strict energy constraint where nodes remain in a deep sleep state and activate solely to transmit data packets according to a stochastic duty cycle. Consequently, the network operates without centralized synchronization or wake-up receiver infrastructure.

This design choice reflects the practical reality of energy-harvesting IoT deployments. Class A devices are the de facto standard in real-world LoRaWAN networks because their simplicity and low power consumption make them ideal for battery-powered and energy-harvesting

applications. The lack of downlink capability in Class A devices precludes the use of wake-up receiver architectures, which would require active, power-consuming receivers to remain listening for network-initiated wake-up signals—a fundamental contradiction with the strict energy budgets of sensor nodes. Thus, our modeling choice ensures that the proposed framework remains applicable to genuine large-scale deployments, rather than idealized systems with unlimited infrastructure support.

Data Generation and Buffer Model: Data generation at each sensor $i \in \mathcal{N}$ follows a *Poisson process* with rate λ_i:

$$P\left(\text{packet generated at sensor } i \text{ in interval } [t, t+\Delta t]\right) = \lambda_i \cdot \Delta t \tag{9}$$

Generated packets accumulate in finite transmission buffers with capacity B_{max}:

$$\text{Buffer}_i(t) \in [0, B_{\text{max}}] \tag{10}$$

Once a buffer reaches capacity, newly generated packets are discarded without transmission. This models the realistic scenario wherein sensor data loses value if not collected promptly—a behaviour common in time-sensitive monitoring applications such as environmental sensing, disaster response, and industrial monitoring. The buffer overflow mechanism creates a hard deadline for data collection: if the UAV does not visit a sensor before its buffer fills, the oldest or newest packets (depending on buffer policy) are lost permanently.

Mission Implications: The combination of Class A asynchronous operation and finite buffer capacity creates a challenging multi-objective optimization problem for the DRL agent:

- **Fairness constraint:** The agent must visit all sensors periodically to prevent buffer overflow and information loss.
- **Energy efficiency constraint:** The agent must minimize flight distance and hovering time to conserve battery energy.

- **Throughput objective:** The agent must maximize the total number of packets collected within the mission horizon.

These competing objectives naturally emerge from the Class A device constraints and form the basis of the reward function design discussed in Section **??**.

4.1.5 Collision Resolution via Capture Effect

While orthogonal spreading factors do not interfere with one another, *intra-SF collisions* occur when multiple sensors attempt transmission on the same spreading factor and frequency simultaneously. We resolve these collisions using the *Capture Effect model*, which exploits the power-law properties of LoRa receivers to successfully decode the strongest signal whilst suppressing weaker interferers.

Capture Effect Criterion: The gateway successfully decodes a signal from sensor i if its received power $P_{r,i}$ exceeds the sum of all interfering signals by a capture threshold τ_{cap} (typically 6 dB):

$$\frac{P_{r,i}}{\sum_{j \in I_{\text{co-channel}}} P_{r,j} + N_0} \geq \tau_{\text{cap}} \tag{11}$$

where N_0 is the background noise power spectral density and $I_{\text{co-channel}} = \{j \in \mathcal{N} \mid j \neq i, \text{SF}_j(t) = \text{SF}_i(t)\}$ denotes the set of sensors transmitting on the same spreading factor as sensor i.

Implication for Trajectory Optimization: This constraint is critical to the DRL agent's learning objective. The Capture Effect forces the agent to optimize its trajectory not solely for *coverage*—i.e., achieving minimum received signal strength indicator (RSSI) thresholds—but rather to *induce a power differential* that ensures successful packet capture during congestion events.

Consider an intra-SF collision scenario in which two sensors simultaneously transmit on the same spreading factor. The received powers at the UAV are:

$$P_{r,i} = P_t G_t G_r \frac{h_t^2 h_r^2}{d_{i,t}^4} \tag{12}$$

$$P_{r,j} = P_t G_t G_r \frac{h_t^2 h_r^2}{d_{j,t}^4} \tag{13}$$

For sensor i to be successfully captured, the UAV must position itself such that $d_{i,t} < d_{j,t}$, thereby satisfying:

$$\frac{d_{j,t}^4}{d_{i,t}^4} \geq \tau_{\text{cap}} \tag{14}$$

This creates a **non-trivial spatial optimization problem** wherein naive distance-minimization or greedy path-planning strategies fundamentally fail. The agent must learn to position itself strategically closer to the desired sensor during collision events, not merely visit all sensors with equal frequency. This insight is fundamental to the work: the UAV's physical position becomes an *active control variable* for the communication protocol, rather than a passive design parameter. Unlike static path planning, which treats the LoRaWAN protocol as a fixed constraint, the DRL framework allows the agent to *manipulate* protocol outcomes through intelligent positioning.

4.1.6 Gateway Architecture

The UAV communication module is modeled as a *single-channel multi-spreading-factor gateway*. We exploit the orthogonality of LoRa spreading factors to demodulate multiple concurrent signals transmitted simultaneously on distinct spreading factors. This property—unique to LoRa modulation—allows us to achieve concurrent channel capacity beyond what would be possible in traditional narrowband systems.

Spreading Factor Selection: Rather than implementing the full LoRaWAN standard set, which defines spreading factors SF $\in \{7, 8, 9, 10, 11, 12\}$, we restrict the operational spreading factor set to:

$$\mathcal{S} = \{7, 9, 11, 12\} \tag{15}$$

This pragmatic restriction reflects the hardware limitations of real unmanned aerial vehicle (UAV)-mounted gateways, which typically operate a single transceiver with time-division or frequency-division multiplexing across a restricted subset of spreading factors. The specific selection of $\mathcal{S} = \{7, 9, 11, 12\}$ maintains sufficient spectral diversity to prevent the *spreading factor monoculture* whilst remaining implementable on lightweight airborne platforms with constrained power and computational budgets.

The four spreading factors in $\mathcal{S}$ are mutually orthogonal, meaning signals modulated with different spreading factors can theoretically coexist on the same frequency channel without interfering with one another, provided they do not collide on the same spreading factor and frequency pair.

Concurrent Reception and Collision Handling: This single-channel multi-SF architecture enables the UAV to receive up to four simultaneous packets, subject to the constraint:

$$|\{i \in \mathcal{N} : \text{SF}_i(t) = k\}| \leq 1 \quad \forall k \in \mathcal{S} \tag{16}$$

where $\text{SF}_i(t)$ denotes the spreading factor assigned to sensor i at time t, and $\mathcal{N} = \{1, \ldots, N\}$ is the set of all deployed sensor nodes.

Constraint ((16)) ensures that at most one sensor transmits on each spreading factor at any given time. Packets that violate this constraint—i.e., multiple sensors transmitting on the same spreading factor and frequency simultaneously—are subject to collision resolution via the Capture Effect mechanism detailed in Section **??**.

4.2 Problem Formulation

We formulate the UAV trajectory optimization and resource allocation problem as a maximization of the total collected data throughput, subject to energy, kinematic, and protocol constraints.

The optimization problem is defined as follows:

$$(\mathcal{P}_1): \quad \max_{\pi} \quad \sum_{t=0}^{T} \sum_{i=1}^{N} \mathcal{D}_{i,t} \tag{17}$$

$$\text{s.t.} \quad RSSI_{i,t}(p_t) \geq RSSI_{min}(SF_{i,t}), \quad \forall i \in \mathcal{N} \tag{18}$$

$$E_{t+1} = E_t - P(a_t) \cdot \Delta t \geq 0, \quad \forall t \tag{19}$$

$$\sum_{i \in \mathcal{N}} \mathbb{I}(SF_{i,t} = k) \leq 1, \quad \forall k \in \{7, \ldots, 12\} \tag{20}$$

$$p_{t+1} = p_t + v(a_t) \cdot \Delta t \tag{21}$$

$$p_0 = p_{start}, \quad p_T = p_{end} \tag{22}$$

Where:

- $\mathcal{N}$: Set of N IoT sensor devices.
- π: The control policy mapping states to actions a_t.
- $\mathcal{D}_{i,t}$: Data collected from sensor i at time t (1 if successful, 0 otherwise).
- p_t: UAV position $[x_t, y_t, z_t]^T$ at time t.
- $RSSI_{i,t}$: Signal strength received from sensor i, calculated via the Two-Ray model.
- $SF_{i,t}$: Spreading Factor assigned to sensor i by the ADR model.
- E_t: Remaining UAV battery energy.
- $\mathbb{I}(\cdot)$: Indicator function for collision detection (enforcing SF orthogonality).

This formulation captures the non-convex nature of the problem, where the decision variable (UAV position p_t) non-linearly determines the link quality ($RSSI$) and protocol state (SF), necessitating a Deep Reinforcement Learning approach.

4.2.1 State Space Definition

The state space $\mathcal{S}$ is designed to provide the agent with effective awareness of the network conditions and UAV state. At time step t, the state vector $S_t \in \mathcal{S}$ is defined as a tuple:

$$S_t = [p_t, E_t, \mathbf{B}_t, \mathbf{U}_t] \tag{23}$$

where:

- $p_t = [x_t, y_t]^T$: The 2D horizontal position of the UAV in the grid. The altitude is assumed constant at h_u meters, simplifying the control problem to horizontal trajectory optimization. This design choice isolates the effects of path planning from altitude dynamics.
- $E_t \in [0, E_{\text{max}}]$: The remaining battery energy of the UAV (in joules or watt-hours). This constraint ensures the agent learns time-efficient trajectories within finite energy budgets.
- $\mathbf{B}_t \in [0, B_{\text{max}}]^N$: A continuous vector representing the current data buffer occupancy level of each sensor $i \in \mathcal{N}$. Rather than binary status (full/empty), we observe the actual buffer level in bytes, providing fine-grained information about imminent overflow risk. This allows the agent to make nuanced decisions about which sensors require urgent attention.
- $\mathbf{U}_t \in [0, 1]^N$: A normalized *Urgency* vector derived from each sensor's data loss rate and buffer utilization. The urgency of sensor i is defined as:

$$U_{i,t} = \frac{B_{i,t}}{B_{\text{max}}} \cdot \frac{\text{data_generation_rate}_i}{\text{collection_rate}_i} \tag{24}$$

  This metric captures the *immediate risk* of buffer overflow: sensors with high buffer occupancy and fast data generation rates have high urgency. Unlike classical Age of Information (which only increases with time), urgency provides a *denser reward signal* tied directly to packet loss probability.

Dimensionality and Design Justification: The total dimensionality of the state space is $3 + 2N$, where:

$$\dim(\mathcal{S}) = \underbrace{2}_{\text{position}} + \underbrace{1}_{\text{battery}} + \underbrace{N}_{\text{buffers}} + \underbrace{N}_{\text{urgency}} = 3 + 2N \tag{25}$$

This represents a significant reduction from the theoretical maximum of $5 + 3N$ (if we included 3D position, binary buffer status, and spreading factors). Our design choices prioritize computational efficiency while maintaining sufficient observability:

1. **2D vs 3D Position:** We fix altitude to a constant h_u, reducing positional dimensionality from 3 to 2. This is justified because the Two-Ray Ground Reflection model (Eq. **??**) shows that height affects path loss through a quadratic term; thus, optimizing vertical position would require balancing coverage against energy cost. For a BEng project, horizontal planning suffices to demonstrate protocol-aware navigation.

2. **Urgency as Proxy for AoI:** Rather than explicitly observe Age of Information (which would require tracking timestamp history), we use buffer occupancy and generation rate to infer urgency. This is physically motivated: a sensor with a nearly-full buffer is *more urgent* than a sensor with empty buffer, regardless of how long data has been waiting. This provides immediate, actionable feedback for the agent.

3. **Spreading Factors Not Observed:** While $\mathbf{SF}_t \in \{7, \ldots, 12\}^N$ is not explicitly in the observation vector, the agent learns to *infer* spectral congestion through the dynamic changes in $\mathbf{U}_t$ and $\mathbf{B}_t$. When a sensor is stuck on SF12 (slow rate), its urgency will rise rapidly relative to nearby sensors. This implicit representation reduces dimensionality by an order of N while preserving the agent's ability to detect and avoid SF Monoculture.

Frame Stacking for Temporal Context: To compensate for the removal of explicit SF observation, we employ **frame stacking** with $k = 4$ observations. This provides the agent with a temporal window of the last 4 steps, allowing it to observe:

- **Urgency trends:** Is a sensor's urgency rising (approaching overflow) or stable?
- **Collection effectiveness:** Are buffers decreasing after visits, or staying constant?
- **Implied SF dynamics:** Rapidly rising urgency despite proximity suggests SF12 congestion.

The stacked state vector is:

$$S_t^{\text{stacked}} = [S_{t-3}, S_{t-2}, S_{t-1}, S_t] \in \mathbb{R}^{4(3+2N)} \tag{26}$$

This transformation makes the environment effectively Markovian despite being partially observable (SF is not directly observed), enabling the DQN agent to infer temporal patterns and make decisions based on network dynamics rather than instantaneous snapshots.

Summary: The state space design represents a *principled trade-off* between observability and dimensionality. By using urgency as a proxy for AoI and inferring spectral state from buffer dynamics, we achieve:

- **Computational efficiency:** $3 + 2N$ dimensions per observation, feasible for $N = 20$ sensors.
- **Physical grounding:** Urgency directly measures packet loss risk (not just staleness).
- **Implicit protocol awareness:** SF congestion is reflected in urgency trends, detectable via frame stacking.

4.2.2 Action Space Definition

The action space $\mathcal{A}$ is discrete to facilitate the use of the Deep Q-Network (DQN) algorithm. At each time step, the agent selects one action $a_t \in \mathcal{A}$:

$$\mathcal{A} = \{\text{Move}_N, \text{Move}_S, \text{Move}_E, \text{Move}_W, \text{Hover}\} \tag{27}$$

- **Movement Actions:** The four cardinal movement actions update the UAV's horizontal position by a fixed step size δ_d (e.g., 1 meter). The altitude is assumed fixed in this study to isolate the trajectory optimization problem.
- **Hover Action:** The `Hover` action maintains the UAV's current position ($p_{t+1} = p_t$). This action is energetically expensive per unit distance (zero displacement) but is operationally essential. It allows the UAV to maintain a stable channel long enough to transmit large packets (high Time-on-Air) or to wait for a sensor's duty cycle window to open.

4.3 Sensor Placement in Simulation

4.3.1 Stochastic Sensor Deployment

The spatial distribution of the N sensor nodes is modeled using a continuous uniform distribution to simulate an unstructured, ad-hoc network topology. For a deployment area defined by dimensions $L \times W$, the coordinates (x_i, y_i) for the i-th sensor are sampled independently:

$$x_i \sim \mathcal{U}(0, L), \quad y_i \sim \mathcal{U}(0, W) \tag{28}$$

The probability density function (PDF) for a sensor appearing at any specific coordinate x along a dimension is given by:

$$f(x) = \frac{1}{b - a} \tag{29}$$

where a and b represent the lower and upper spatial bounds, respectively. For our simulation grid of 50×50 units (where $a = 0, b = 50$), the uniform probability density is constant at $f(x) = \frac{1}{50} = 0.02$.

This stochastic initialization serves three critical research functions:

1. It ensures the DRL agent learns a generalizable navigation policy rather than memorizing a static map.
2. mimics realistic IoT deployment scenarios where sensor locations may not be grid-aligned.
3. Random placement naturally creates density clusters, inducing the “SF Monoculture” problem that the agent must learn to resolve.

4.3.2 Multi-Objective Reward Function

To facilitate efficient learning in this long-horizon optimization problem, we employ a dense, multi-objective reward function. A purely sparse reward (e.g., rewarding only upon full mission completion) would result in a vanishing gradient problem given the large state space.

Instead, the reward function R_t at time step t is designed as a weighted sum of competing objectives, providing the agent with immediate feedback on the quality of its actions.

The total reward is defined as:

$$R_t = w_{coll} \cdot R_{coll} - w_{energy} \cdot C_{energy} - w_{AoI} \cdot C_{AoI} - w_{hover} \cdot C_{hover} \tag{30}$$

where w_i are tunable hyperparameters balancing the relative importance of each term.

1. Data Collection Reward (R_{coll}): This is the primary positive reinforcement. The agent receives a discrete reward for every unique data packet successfully decoded from a sensor:

$$R_{coll} = \sum_{i=1}^{N} \mathbb{I}(\text{Packet}_i \text{ received at } t) \tag{31}$$

This term encourages the agent to position itself in regions of high spectral efficiency where the LoRaWAN “Capture Effect” can resolve collisions.

2. Energy Consumption Penalty (C_{energy}): To enforce energy efficiency, a penalty is applied proportional to the battery drained by the chosen action:

$$C_{energy} = P(a_t) \cdot \Delta t \tag{32}$$

This term discourages inefficient, meandering paths and incentivizes the agent to complete the collection task using the minimum necessary flight time.

3. Age of Information Penalty (C_{AoI}): To ensure network fairness and data freshness, a penalty is applied based on the cumulative Age of Information of all sensors:

$$C_{AoI} = \frac{1}{N} \sum_{i=1}^{N} AoI_{i,t} \tag{33}$$

This critical term prevents the agent from ignoring distant or difficult-to-reach sensors. It forces the UAV to prioritize “stale” nodes, thereby preventing information starvation.

4. Hovering Regularization (C_{hover}): While hovering is necessary for LoRaWAN connectivity, excessive hovering is a local optimum trap. A small fixed penalty is applied to non-

movement actions to encourage active exploration when data reception is not imminent:

$$C_{hover} = \mathbb{I}(a_t = \text{HOVER}) \cdot \lambda_{hover} \tag{34}$$

This shaped reward structure transforms the sparse “mission success” objective into a dense, step-by-step gradient that guides the agent toward the optimal policy.

4.4 Deep Reinforcement Learning Implementation

4.4.1 Deep Q-Network (DQN) Architecture

The agent’s decision-making policy is approximated using a Deep Q-Network (DQN). We employ a Multi-Layer Perceptron (MLP) architecture to map the high-dimensional state vector S_t to Q-values for the five discrete actions.

The network consists of an input layer matching the state dimension ($4 + 3N$, where N is the number of sensors), followed by two fully connected hidden layers with 64 neurons each. To introduce non-linearity, Rectified Linear Unit (ReLU) activation functions are applied after each hidden layer. The output layer consists of 5 linear neurons, each corresponding to the estimated Q-value $Q(s, a)$ for a specific action in $\mathcal{A}$.

The network weights θ are optimized using the Adam optimizer to minimize the temporal difference error between the predicted Q-values and the target Q-values derived from the Bellman equation. To ensure training stability, a separate target network $\hat{Q}$ with parameters θ^- is maintained and updated periodically (every C steps).

4.4.2 Custom Gymnasium Environment Design

The simulation environment incorporates a probabilistic link-layer model to move beyond binary ‘in-range’ abstractions. Each sensor’s transmission success is determined by the product of its protocol-mandated duty cycle and a sigmoid-mapped signal-to-noise ratio (SNR). This mapping calculates the instantaneous success probability P_{overall} by comparing the RSSI which is derived from the Two-Ray Ground Reflection model against the noise

floor and the specific SNR requirements of the current Spreading Factor. By setting a transmission threshold (e.g., $P_{\text{overall}} > 0.5$), the environment forces the Deep Q-Network agent to navigate the trade-off between proximity-induced signal strength and the stochastic availability of sensor nodes. This in turn allows for the creation and editing of link quality.

4.5 Architecture and Interface

The simulation is architected as a custom environment compliant with the Gymnasium (formerly OpenAI Gym) interface, facilitating seamless integration with standard DRL libraries such as Stable-Baselines3. The environment encapsulates the complex cyber-physical interactions into a discrete time-stepping logic.

The core logic resides in the `step(action)` function, which executes the following sequence at every iteration:

1. **Physics Update:** The UAV's position p_t is updated based on the selected motion primitive, and battery energy E_t is decremented according to the power consumption model.
2. **Channel Propagation:** The Euclidean distances to all sensors are recalculated. The Two-Ray Ground Reflection model is applied to determine the instantaneous RSSI for every ground node.
3. **Protocol Logic:** The internal states of the sensors are updated. This includes incrementing the Age of Information (AoI) counters, generating new data packets based on a Poisson process, and updating the moving average of RSSI values to simulate ADR latency.
4. **Collision Resolution:** The environment checks for packet collisions. If multiple sensors transmit on the same Spreading Factor (SF), the Capture Effect logic determines if the strongest signal (closest sensor) exceeds the interference threshold.
5. **Reward Calculation:** The multi-objective reward R_t is computed based on the outcome of the data collection attempts and the UAV's energy usage.

This modular design ensures that the DRL agent is trained against a rigorous model of the physical and protocol layers, rather than a simplified abstraction.

4.5.1 Training Algorithm

The complete training procedure, which integrates the physics-based updates (Two-Ray model) and protocol constraints (ADR latency) with the DRL agent's decision process, is detailed in Algorithm 1.

Algorithm 1 Protocol-Aware DQN Training for UAV Data Collection (Consolidated)

1: **Initialize:** UAV position p_0, Replay Buffer $\mathcal{D}$
2: **Initialize:** Q-Network $Q(s, a \mid \theta)$, Target Network $Q'(s, a \mid \theta^-)$
3: **Initialize:** Hyperparameters: ϵ_{max}, ϵ_{min}, γ, α, B, C
4: **Initialize:** Sensor states (Buffers, AoI, SF), $\epsilon \leftarrow \epsilon_{\text{max}}$
5: **for** episode $\in [1, M]$ **do**
6: Reset environment: $s_0 \leftarrow (p_{\text{start}}, E_{\text{max}}, \dots)$
7: $t \leftarrow 0$
8: **while** $E > 0$ **and** Data Remaining **do**
9: **Action Selection:**
10: **if** random$() < \epsilon$ **then**
11: $a_t \leftarrow$ RandomAction // Explore
12: **else**
13: $a_t \leftarrow \arg\max_a Q(s_t, a; \theta)$ // Exploit
14: **end if**
15: **Step Environment and Observe:**
16: $s_{t+1}, r_t,$ done $\leftarrow$ Environment.Step(a_t)
17: // Step internally performs physics, protocol update, data collection, and reward calc.
18: **Store Transition:**
19: Store (s_t, a_t, r_t, s_{t+1}) in $\mathcal{D}$
20: **Training Step:**
21: **if** $|\mathcal{D}| \geq B$ **then**
22: Sample random minibatch $\{(s_j, a_j, r_j, s'_j)\}_{j=1}^{B}$ from $\mathcal{D}$
23: Compute TD target: $y_j \leftarrow r_j + \gamma \max_{a'} Q'(s'_j, a' \mid \theta^-)$
24: Compute loss: $L(\theta) \leftarrow \frac{1}{B} \sum_{j=1}^{B} (y_j - Q(s_j, a_j \mid \theta))^2$
25: Perform gradient descent: $\theta \leftarrow \theta - \alpha \nabla_\theta L(\theta)$
26: **end if**
27: $s_t \leftarrow s_{t+1}$, $t \leftarrow t + 1$
28: **end while**
29: **Epsilon Decay:**
30: $\epsilon \leftarrow \max(\epsilon_{\text{min}}, \epsilon - \frac{\epsilon_{\text{max}} - \epsilon_{\text{min}}}{M})$
31: **Update Target Network:**
32: **if** $t \bmod C = 0$ **then**
33: $\theta^- \leftarrow \theta$
34: **end if**
35: **end for**

In the consolidated format (Algorithm 1), the environment's complexity is contained within the Environment.Step(a_t) call (Line 19). This single operation sequentially performs the kinematic update, computes the RSSI via the Two-Ray Ground Model, executes the ADR

protocol logic, determines packet capture based on the Capture Effect, and calculates the final multi-objective reward signal before transitioning the system to state s_{t+1}.

4.5.2 Hyperparameters

The hyperparameters used for training the DQN agent were selected based on preliminary tuning experiments to balance sample efficiency with stability. The specific values are detailed in Table 1.

Table 1. DQN Hyperparameters and Training Configuration

Parameter	**Value**
Total Timesteps	1×10^6
Learning Rate (α)	1×10^{-4}
Discount Factor (γ)	0.99
Buffer Size	100,000 transitions
Batch Size	32
Target Update Interval (C)	1000 steps
Exploration Strategy	ϵ-greedy
Initial ϵ	1.0
Final ϵ	0.05
Exploration Fraction	10% of total steps
Network Architecture	MLP $[512, 512, 256]$
Activation Function	ReLU

4.6 Summary

5 Results and discussion

5.1 Introduction

This chapter evaluates the performance of the proposed protocol-aware Deep Reinforcement Learning (DRL) agent. To validate the framework, we conduct a comparative analysis

against two standard path-planning heuristics: a Nearest Neighbor approach and a sophisticated SF-Aware Max-Throughput strategy. The performance is quantified in terms of data collection efficiency, network coverage, and energy consumption within the high-fidelity Gymnasium simulation.

5.2 Experimental Setup and Baselines

To rigorously evaluate the efficacy of the proposed DRL agent, its performance is benchmarked against two standard heuristics. These baselines represent the current "status quo" for UAV path planning.

5.2.1 Baseline 1: Nearest Sensor Greedy

The "Nearest Sensor" algorithm (Algorithm 2) represents the standard "naive" path planning approach. At each time step, the UAV identifies all sensors with data in their buffers and selects the one with the minimum Euclidean distance from its current position. This baseline minimizes flight time between targets but is "protocol-blind," ignoring the impact of Spreading Factors (SF) and potential network congestion.

Algorithm 2 Baseline 1: Nearest Sensor Greedy Strategy

Input: UAV position p_{uav}, Set of Sensors $\mathcal{N}$
Output: Selected Action a_t
$S_{candidates} \leftarrow \{s \in \mathcal{N} \mid s.buffer > 0\}$
if $S_{candidates}$ is empty **then**
    return `Hover`
end if
$d_{min} \leftarrow \infty$
$s_{target} \leftarrow$ null
for each sensor s in $S_{candidates}$ **do**
    $d \leftarrow$ distance$(p_{uav}, s.position)$
    if $d < d_{min}$ **then**
        $d_{min} \leftarrow d$
        $s_{target} \leftarrow s$
    end if
end for
$a_t \leftarrow$ MoveTowards(s_{target})
return a_t

5.2.2 Baseline 2: Max-Throughput Greedy (SF-Aware)

The “Max-Throughput” algorithm (Algorithm 3) is a sophisticated heuristic designed to maximize the instantaneous data rate. It prioritizes sensors capable of transmitting at the fastest Spreading Factors (e.g., SF7–SF9), effectively “harvesting” the easiest data first. The UAV will only travel to distant (SF12) sensors if no high-rate options are available. This baseline serves as a strong competitor, representing an expert-designed engineering solution that lacks the long-horizon planning capabilities of the DRL agent.

Algorithm 3 Baseline 2: Max-Throughput (SF-Aware) Strategy

Input: UAV position p_{uav}, Set of Sensors $\mathcal{N}$
Output: Selected Action a_t
Phase 1: Immediate Collection
$S_{fast} \leftarrow \{s \in \mathcal{N} \mid s.buffer > 0$ **and** $SF(s) \leq 9\}$
if S_{fast} is not empty **then**
    return `Hover` ▷ Stop and collect high-rate data
end if
Phase 2: Target Selection
$S_{candidates} \leftarrow \{s \in \mathcal{N} \mid s.buffer > 0\}$
$BestScore \leftarrow -\infty$
$s_{target} \leftarrow$ null
for each sensor s in $S_{candidates}$ **do**
    $SF_Score \leftarrow (13 - s.SF) \times w_1$ ▷ Prioritize SF7
    $Dist_Penalty \leftarrow$ distance$(p_{uav}, s) \times w_2$
    $Score \leftarrow SF_Score - Dist_Penalty$
    if $Score > BestScore$ **then**
        $BestScore \leftarrow Score$
        $s_{target} \leftarrow s$
    end if
end for
$a_t \leftarrow$ MoveTowards(s_{target})
return a_t

5.3 Simulation Results

5.3.1 Training Convergence

5.3.2 Comparative Performance Analysis

5.3.3 Breaking the SF Monoculture

5.3.4 Fairness and Coverage Analysis

5.4 Summary

6 Conclusions and future work

6.1 Conclusions

[1] [3] [4] [2] [5]

6.2 Future work

[1] [3] [4] [2] [5] [6] [7]

References

[1] Z. Wei et al., "Uav-assisted data collection for internet of things: A survey," *IEEE Internet of Things Journal*, vol. 9, no. 17, pp. 15 460–15 483, 2022 (cited on pp. 7, 9, 12, 34).

[2] K. Nakamura et al., "A lora-based protocol for connecting iot edge computing nodes to provide small-data-based services," *Digital Communications and Networks*, vol. 8, no. 3, pp. 257–266, 2022 (cited on pp. 7, 10, 34).

[3] D. Zorbas, A. Sabyrbek, and L. Di Puglia Pugliese, "Revisiting the problem of optimizing spreading factor allocations in lorawan: From theory to practice," *Computer Communications*, vol. 243, p. 108 321, 2025 (cited on pp. 7, 10–12, 34).

[4] R. Xiong et al., “Flyinglora: Towards energy efficient data collection in uav-assisted lora networks,” *Computer Networks*, vol. 220, p. 109 511, 2023 (cited on pp. 7, 10, 11, 34).

[5] D. C. Nguyen et al., “6g internet of things: A comprehensive survey,” *IEEE Internet of Things Journal*, vol. 9, no. 1, pp. 359–383, 2022, Originally arXiv:2108.04973 (cited on pp. 7, 11, 34).

[6] K. Seerangan et al., “A novel energy-efficiency framework for uav-assisted networks using adaptive deep reinforcement learning,” *Scientific Reports*, vol. 14, no. 1, p. 22 188, 2024. DOI: 10.1038/s41598-024-71621-x (cited on p. 34).

[7] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd. Cambridge, MA: The MIT Press, 2018 (cited on p. 34).

Appendices

A Project outline

Project outline as submitted at the start of the project is a required appendix. Put here.

B Risk assessment

Risk assessment is a required appendix. Put here.